

LERNSKRIPT / EINSTIEG / 30 MINI-AUFGABEN

PHP von Anfang an. Ein Schritt nach dem anderen.

Erst einen Wert speichern. Dann etwas ausgeben, prüfen und verändern. Zum Schluss verpackst du bekannte Anweisungen in eigene kleine Funktionen.

SO KLEIN BLEIBEN DIE AUFGABEN

Jede Aufgabe hat ein klares Einzelziel und wenige Anweisungen. Meist reichen etwa drei Codezeilen. Startwerte, Funktionsaufrufe und zusätzliche Klammerzeilen dürfen dazukommen. Lesbarkeit geht vor künstlich verkürztem Code.

Dein Weg durch das Skript

Block	Aufgaben	Thema
A	01-05	Variablen und Kommentare
B	06-10	Datentypen und Rechnen
C	11-15	Entscheidungen
D	16-20	Schleifen und kleine Arrays
E	21-25	Strings verändern
F	26-30	Eigene Funktionen

So startest du mit XAMPP

1. Speichere die PHP-Dateien in C:\xampp\htdocs\PHP \.
2. Starte Apache im XAMPP Control Panel.
3. Öffne die Übungsdatei im Browser über die folgende Adresse:

`http://localhost/PHP /PHP _PHP_Grundlagen_Aufgaben.php`

Bei einem abweichenden Apache-Port muss der Port in der Adresse ergänzt werden.

Drei Dateien, ein Ablauf

Im PDF liest du die Grundlagen und die Arbeitsaufträge. In `PHP _PHP_Grundlagen_Aufgaben.php` ergänzt du deinen Code. Die Musterlösungen stehen ab Seite 14 im PDF und vollständig ausführbar in `PHP _PHP_Grundlagen_Loesungen.php`.

Arbeite Aufgabe für Aufgabe: erst Ausgabe vorhersagen, dann programmieren, ausführen und vergleichen. Formulare, Datenbanken, Sessions, Klassen und aufwendige Eingabeprüfungen kommen noch nicht vor.

BLOCK A / KURZ ERKLÄRT / ZU AUFGABEN 01-05

Variablen und Kommentare

Eine Variable speichert einen Wert

```
$ort = "Bonn";  
echo $ort . PHP_EOL;
```

\$ort ist der Variablenname. = weist den Wert zu. "Bonn" ist ein Textwert; ; beendet die Anweisung. echo gibt aus, . verbindet Texte und PHP_EOL erzeugt einen Zeilenumbruch in unserer Klartextausgabe. [1, 4]

Bezeichner: Was ist erlaubt?

Geeignet	Nicht als normaler Variablenname
\$name, \$kursName	\$1name – beginnt mit einer Ziffer
\$name2, \$_anzahl	\$vor name – enthält ein Leerzeichen
\$vor_name	\$vor-name – enthält einen Bindestrich

Für unsere Namen verwenden wir am Anfang A-Z, a-z oder _; danach sind auch Ziffern erlaubt. \$name und \$Name sind verschieden. Umlaute können technisch funktionieren; wir verwenden bewusst ASCII-Namen ohne Umlaute. \$this ist speziell reserviert. [1]

Kommentare erklären, statt ausgeführt zu werden

```
// Ein Kommentar bis zum Zeilenende  
# Eine weitere Schreibweise für eine Kommentarzeile  
/* Ein Blockkommentar  
   über mehrere Zeilen */
```

Kommentare werden nicht ausgegeben. Blockkommentare mit /* ... */ nicht ineinander verschachteln. [1]

Zuweisen ist nicht Vergleichen

```
$ort = "Bonn";  
$ort = "Ulm";  
echo $ort . PHP_EOL;
```

Die zweite Zuweisung ersetzt den Wert. Ausgegeben wird Ulm. Der Bezeichner bleibt \$ort. [1]

EINFACHES GRUNDGERÜST

Beginne eine neue Datei mit <?php und direkt danach mit declare(strict_types=1);. Ein schließendes PHP-Tag lassen wir in reinen PHP-Dateien weg. Was Strict Types leistet, steht im nächsten Block. [1, 3]

BLOCK A / MINI-AUFGABEN 01-05

Variablen und Kommentare

Namen vergeben, Werte speichern, Texte ausgeben.

01 / Eine Variable anlegen

1. Lege die Variable `$kurs` an und speichere den Text "PHP".
2. Gib den Inhalt der Variable mit `echo` aus.
3. Ergänze mit `PHP_EOL` einen Zeilenumbruch.

Erwartet: PHP

02 / Bezeichner reparieren

1. Korrigiere den ungültigen Namen `$1name` zu `$name1`; Wert: "Mia".
2. Korrigiere den ungültigen Namen `$kurs name` zu `$kursName`; Wert: "PHP".
3. Gib beide Werte mit dem Text " lernt " dazwischen aus.

Erwartet: Mia lernt PHP

03 / Kommentare schreiben

1. Schreibe vor den Code einen Kommentar mit `//`.
2. Speichere "Berlin" in `$stadt`; ergänze dahinter einen Kommentar mit `/* ... */`.
3. Gib `$stadt` aus; ergänze dahinter einen Kommentar mit `#`.

Erwartet: Berlin

04 / Einen Wert überschreiben

1. Speichere die Zahl 5 in `$punkte`.
2. Weise derselben Variable anschließend die Zahl 9 zu.
3. Gib den aktuellen Wert aus.

Erwartet: 9

05 / Texte verbinden

1. Speichere "Mia" in `$vorname` und "Sommer" in `$nachname`.
2. Verbinde die beiden Werte bei der Ausgabe mit dem Punktoperator.
3. Setze zwischen Vorname und Nachname genau ein Leerzeichen.

Erwartet: Mia Sommer

Arbeite direkt im gleich nummerierten Abschnitt der Übungsdatei. Dort findest du auch einen kurzen Hinweis zu jeder Aufgabe.

BLOCK B / KURZ ERKLÄRT / ZU AUFGABEN 06-10

Datentypen und Strict Types

Welche Werte gibt es?

Typ	Bedeutung	Beispiel
int	Ganze Zahl	3
float	Zahl mit Nachkommastellen	2.5
string	Text; auch Ziffern können Text sein	"7"
bool	Wahrheitswert	true, false
array	Sammlung mit Schlüsseln und Werten	["rot", "blau"]
null	Expliziter Nullwert	null
object / resource	Objekt / externe Ressource	Folgen später

In PHP-Code steht bei Dezimalzahlen ein Punkt, kein Komma. `var_dump($wert)` zeigt Typ und Wert; das ist besonders für false und null hilfreich. Bei array ohne angegebene Schlüssel beginnt unser Beispiel mit Index 0. [2]

Das Grundgerüst mit declare

```
<?php
declare(strict_types=1);
```

`declare(strict_types=1);` steht als erste Anweisung. Davor darf keine Ausgabe und kein HTML stehen; speichere als UTF-8 ohne BOM. Kommentare vor der Anweisung wären erlaubt. [3]

Strict Types betrifft skalare Typdeklarationen, zum Beispiel `int` oder `string` bei Funktionsparametern und Rückgaben. Bei Parametern zählt die aufrufende Datei. Ein `int` ist weiterhin für `float` zulässig. Das sehen wir bei den eigenen Funktionen. [3]

WAS STRICT_TYPES NICHT TUT

Es macht normale Variablen nicht fest typisiert und ist keine Inhaltsprüfung. `$wert = 5;` und anschließend `$wert = "Text";` ist weiterhin erlaubt. Auch eine negative Zahl kann ein gültiger `int` sein. [2, 3]

Rechnen und bewusst umwandeln

+ addiert, - subtrahiert, * multipliziert, / dividiert. % liefert den Rest einer Ganzzahldivision, zum Beispiel `7 % 3` ergibt 1. (int) "12" wandelt den bekannten Zahlenstring in 12 um. Ein Cast ist keine Prüfung beliebiger Eingaben. [2, 4]

BLOCK B / MINI-AUFGABEN 06-10

Datentypen und Rechnen

Wert und Typ unterscheiden; mit bekannten Werten rechnen.

06 / int und float ansehen

1. Speichere die ganze Zahl 3 in \$anzahl.
2. Speichere die Dezimalzahl 2.5 in \$preis.
3. Zeige beide Werte samt Datentyp mit einem Aufruf von var_dump().

Erwartet: int(3) und float(2.5)

07 / string und bool unterscheiden

1. Speichere den Text "7" in \$zahlAlsText.
2. Speichere den Wahrheitswert true in \$aktiv.
3. Zeige beide Werte samt Datentyp mit var_dump().

Erwartet: string(1) "7" und bool(true)

08 / null und ein kleines Array

1. Lege \$hinweis mit dem Wert null an.
2. Lege \$farben als Array mit "rot" und "blau" an.
3. Zeige \$hinweis und das erste Arrayelement mit var_dump().

Erwartet: NULL und string(3) "rot"

09 / Mit Variablen rechnen

Start: \$preis = 4; \$anzahl = 3;

1. Multipliziere \$preis mit \$anzahl.
2. Speichere das Ergebnis in \$gesamt.
3. Gib \$gesamt aus.

Erwartet: 12

10 / Einen bekannten Zahlenstring umwandeln

Start: \$eingabe = "12";

1. Wandle den vorgegebenen String mit (int) in eine ganze Zahl um.
2. Speichere das Ergebnis in \$zahl.
3. Prüfe das Ergebnis mit var_dump().

Erwartet: int(12)

Arbeite direkt im gleich nummerierten Abschnitt der Übungsdatei. Dort findest du auch einen kurzen Hinweis zu jeder Aufgabe.

BLOCK C / KURZ ERKLÄRT / ZU AUFGABEN 11-15

Entscheidungen treffen

Eine Bedingung - ein oder mehrere Wege

```
if ($wert > 10) {  
    echo "gross" . PHP_EOL;  
} elseif ($wert === 10) {  
    echo "genau zehn" . PHP_EOL;  
} else {  
    echo "kleiner" . PHP_EOL;  
}
```

Das Beispiel setzt voraus, dass \$wert vorher eine ganze Zahl erhalten hat. if prüft zuerst; elseif prüft sonst weiter; else fängt die übrigen Fälle auf. Ausgeführt wird höchstens der erste zutreffende Zweig. [4]

Die wichtigsten Operatoren

Zeichen	Bedeutung
=	Wert zuweisen, nicht vergleichen
=== / !==	Gleicher Wert und Typ / nicht identisch
<, >, <=, >=	Kleiner, größer, höchstens, mindestens
&& / / !	UND / ODER / NICHT

== erlaubt beim Vergleich Typumwandlungen. Für gleichartige Werte nutzen wir in diesen Übungen bewusst ===. Beispiel: 7 === "7" ist false. [4]

switch: einen bestimmten Wert behandeln

```
switch ($befehl) {  
    case "hilfe":  
        echo "Hilfe anzeigen" . PHP_EOL;  
        break;  
    default:  
        echo "Kein passender Befehl" . PHP_EOL;  
}
```

Auch \$befehl braucht vorher einen Wert. break beendet den passenden Fall; default behandelt sonstige Werte. switch vergleicht nicht strikt; strict_types ändert daran nichts. Wir verwenden hier ausschließlich Textbefehle. [4]

TYPISCHER FEHLER

Kein Semikolon direkt nach if (...) setzen. Der dazugehörige Code gehört in { ... }. Die Klammern bleiben zur besseren Lesbarkeit sichtbar.

BLOCK C / MINI-AUFGABEN 11-15

Entscheidungen

Mit if, elseif, else und switch zwischen Wegen wählen.

11 / Eine negative Zahl korrigieren

Start: `$zahl = -4;`

1. Prüfe mit if, ob `$zahl` kleiner als 0 ist.
2. Setze `$zahl` nur in diesem Fall auf 0.
3. Gib `$zahl` nach der Verzweigung aus.

Erwartet: 0

12 / Zwischen zwei Ausgaben wählen

Start: `$punkte = 7;`

1. Prüfe mit if, ob `$punkte` mindestens 5 beträgt.
2. Gib dann "Ziel erreicht" aus.
3. Gib im else-Zweig "Weiter ueben" aus.

Erwartet: Ziel erreicht

13 / Drei Fälle unterscheiden

Start: `$zahl = 0;`

1. Gib bei einer Zahl kleiner als 0 den Text "negativ" aus.
2. Prüfe danach mit elseif auf `=== 0` und gib "null" aus.
3. Gib im else-Zweig "positiv" aus.

Erwartet: null

14 / Zwei Bedingungen verbinden

Start: `$punkte = 8; $abgegeben = true;`

1. Prüfe, ob `$punkte` mindestens 5 beträgt.
2. Prüfe zugleich, ob `$abgegeben === true` ist. Verbinde beide Prüfungen mit `&&`.
3. Gib nur dann "Bestanden" aus. Ein else-Zweig ist nicht nötig.

Erwartet: Bestanden

15 / Eine Aktion mit switch auswerten

Start: `$aktion = "start";`

1. Werte `$aktion` mit switch aus.
2. Gib im case "start" den Text "Los" aus und beende den Fall mit break.
3. Gib im default-Zweig für alle anderen Werte "Unbekannt" aus.

Erwartet: Los

Arbeite direkt im gleich nummerierten Abschnitt der Übungsdatei. Dort findest du auch einen kurzen Hinweis zu jeder Aufgabe.

BLOCK D / KURZ ERKLÄRT / ZU AUFGABEN 16-20

Wiederholen mit Schleifen

for: mit einem Zähler arbeiten

```
for ($i = 0; $i < 2; $i++) {  
    echo $i . PHP_EOL;  
}
```

Startwert; Bedingung; Änderung. `$i++` erhöht nach jedem Durchlauf um 1. Die Ausgabe ist hier 0, danach 1. [4]

while und do-while: wann wird geprüft?

```
$rest = 2;  
while ($rest > 0) {  
    $rest--;  
}  
  
$wert = 8;  
do {  
    $wert++;  
} while ($wert < 3);
```

`while` prüft vor dem Block und kann ihn ganz überspringen. `do-while` prüft danach und läuft mindestens einmal. Hier endet `$rest` bei 0; `$wert` wird einmal auf 9 erhöht. Das Semikolon hinter `while (...)`; bei `do-while` gehört dazu. [4]

foreach: Werte oder Schlüssel und Werte lesen

```
$liste = ["A", "B"];  
foreach ($liste as $wert) {  
    echo $wert . PHP_EOL;  
}  
  
$lager = ["Block" => 2];  
foreach ($lager as $artikel => $bestand) {  
    $lager[$artikel] = $bestand + 1;  
}
```

Im ersten Beispiel ist `$wert` nacheinander "A" und "B". Im zweiten sind "Block" der Schlüssel und 2 der Wert. Die Änderung muss über `$lager[$artikel]` ins Array zurückgeschrieben werden. [2, 4]

SCHLEIFEN MÜSSEN ENDEN

Prüfe bei Zählschleifen, ob die Bedingung irgendwann falsch werden kann. `break` beendet eine Schleife; `continue` überspringt den Rest des aktuellen Durchlaufs. Beides brauchen wir in diesen fünf Mini-Aufgaben noch nicht. [4]

BLOCK D / MINI-AUFGABEN 16-20

Schleifen und kleine Arrays

Anweisungen wiederholen und Arraywerte gezielt verändern.

16 / Mit for zählen

1. Starte eine for-Schleife mit `$i = 1`.
2. Wiederhole, solange `$i <= 3` gilt; erhöhe `$i` mit `$i++`.
3. Gib `$i` in jedem Durchlauf in einer eigenen Zeile aus.

Erwartet: 1, 2, 3 - jeweils in einer eigenen Zeile

17 / Mit while herunterzählen

Start: `$rest = 3;`

1. Wiederhole mit while, solange `$rest` größer als 0 ist.
2. Gib zuerst den aktuellen Wert aus.
3. Verringere `$rest` danach mit `$rest--` um 1.

Erwartet: 3, 2, 1 - jeweils in einer eigenen Zeile

18 / do-while führt zuerst aus

Start: `$zahl = 5;`

1. Gib innerhalb eines do-Blocks `$zahl` aus.
2. Erhöhe `$zahl` anschließend mit `$zahl++`.
3. Wiederhole am Ende mit while (`$zahl < 3`);. Erkläre die einmalige Ausgabe.

Erwartet: 5 - genau einmal

19 / Arraywerte mit foreach lesen

Start: `$farben = ["rot", "gruen", "blau"];`

1. Durchlaufe `$farben` mit foreach.
2. Nenne die Variable für den jeweiligen Wert `$farbe`.
3. Gib jede Farbe in einer eigenen Zeile aus.

Erwartet: rot, gruen, blau - jeweils in einer eigenen Zeile

20 / Mit Schlüssel und Wert verändern

Start: `$lager = ["Stift" => 2, "Heft" => 0];`

1. Durchlaufe `$lager` mit foreach; verwende `$artikel` als Schlüssel und `$bestand` als Wert.
2. Speichere für jeden Artikel `$bestand + 1` zurück in `$lager[$artikel]`.
3. Gib danach die Bestände von "Stift" und "Heft" getrennt durch " / " aus.

Erwartet: 3 / 1

Arbeite direkt im gleich nummerierten Abschnitt der Übungsdatei. Dort findest du auch einen kurzen Hinweis zu jeder Aufgabe.

BLOCK E / KURZ ERKLÄRT / ZU AUFGABEN 21-25

Strings gezielt verändern

Eine Funktion liefert ein Ergebnis zurück

```
$text = " Tor ";  
$text = trim($text);  
echo $text . PHP_EOL;
```

Ablauf: aktuellen Wert übergeben, Ergebnis zurückbekommen, Ergebnis speichern. Der bloße Aufruf `trim($text)`; überschreibt `$text` nicht. Erst die Zuweisung hält den veränderten Text fest. [5]

Diese fünf Werkzeuge genügen zunächst

Funktion	Wirkung	Beispielergebnis
<code>trim(\$text)</code>	Äußere Leerzeichen und bestimmte weitere Randzeichen entfernen	" Tor " → "Tor"
<code>strtolower(\$text)</code>	ASCII-Buchstaben klein schreiben	"TOR" → "tor"
<code>strtoupper(\$text)</code>	ASCII-Buchstaben groß schreiben	"tor" → "TOR"
<code>strlen(\$text)</code>	Anzahl der Bytes ermitteln	"Tor" → 3
<code>str_replace(...)</code>	Passende Textstellen ersetzen	"alt" → "neu"

Reihenfolge bei der Ersetzung: `str_replace("alt", "neu", $text)`. Alle diese Funktionen werden in den nächsten Aufgaben zunächst einzeln benutzt. [5]

WICHTIG BEI UMLAUTEN UND EMOJIS

`strlen()` zählt Bytes, nicht allgemein sichtbare Zeichen. Bei unseren ASCII-Texten ist beides gleich. Für andere Schriften, Umlaute oder Emojis müssen später passende Mehrbyte-Funktionen und deren Zählweise betrachtet werden. Auch `strtolower()` und `strtoupper()` behandeln hier ASCII-Buchstaben. [5]

Leer ist nicht gleich null oder 0

`$text === ""` prüft genau auf einen leeren String. `trim($text) === ""` erkennt zusätzlich einen String, der nur aus den von `trim()` entfernten Randzeichen besteht. Der String `"0"` ist dabei nicht leer. [4, 5]

Für später: `isset($wert)` bedeutet vorhanden und nicht null. `empty($wert)` ist dagegen auch bei 0 und `"0"` wahr. Verwende es deshalb nicht unüberlegt für gültige Nullmengen. [5]

BLOCK E / MINI-AUFGABEN 21-25

Strings verändern

Vorhandene Funktionen auf Textvariablen anwenden.

21 / Äußere Leerzeichen entfernen

Start: `$name = " Mia ";`

1. Rufe `trim()` mit `$name` auf.
2. Speichere den Rückgabewert wieder in `$name`.
3. Gib den bereinigten Namen aus.

Erwartet: Mia

22 / In Kleinbuchstaben umwandeln

Start: `$text = "PHP MACHT SPASS";`

1. Wandle den Text mit `strtolower()` in Kleinbuchstaben um.
2. Speichere das Ergebnis wieder in `$text`.
3. Gib `$text` aus.

Erwartet: php macht spass

23 / In Großbuchstaben umwandeln

Start: `$text = "hallo php";`

1. Wandle den Text mit `strtoupper()` in Großbuchstaben um.
2. Speichere das Ergebnis wieder in `$text`.
3. Gib `$text` aus.

Erwartet: HALLO PHP

24 / Die Länge eines Textes bestimmen

Start: `$text = "Hallo";`

1. Bestimme die Länge mit `strlen()`.
2. Speichere das Ergebnis in `$laenge`.
3. Gib `$laenge` aus.

Erwartet: 5

25 / Einen Textteil ersetzen

Start: `$text = "Das Auto ist rot.";`

1. Ersetze mit `str_replace()` das Wort "rot" durch "blau".
2. Speichere das Ergebnis wieder in `$text`.
3. Gib den geänderten Satz aus.

Erwartet: Das Auto ist blau.

Arbeite direkt im gleich nummerierten Abschnitt der Übungsdatei. Dort findest du auch einen kurzen Hinweis zu jeder Aufgabe.

BLOCK F / KURZ ERKLÄRT / ZU AUFGABEN 26-30

Eigene Funktionen bauen

Benötigte Werte hinein - Ergebnis heraus

```
function quadrat(int $zahl): int {  
    return $zahl * $zahl;  
}  
  
$ergebnis = quadrat(3);  
echo $ergebnis . PHP_EOL;
```

Hier wird 9 ausgegeben. Die Funktion erhält eine Zahl, berechnet etwas und gibt das Ergebnis an die aufrufende Stelle zurück. Erst echo macht das Ergebnis sichtbar. [6]

Die Bausteine lesen

Teil	Bedeutung
function	Definiert eine eigene Funktion
quadrat	Funktionsname; ohne Dollarzeichen
int \$zahl	Parameter: eine ganze Zahl als Eingabe
: int	Rückgabetyt: eine ganze Zahl als Ergebnis
return ...;	Ergebnis zurückgeben und Funktionsaufruf beenden
quadrat(3)	Aufruf; 3 ist das übergebene Argument

Die Parameter und lokalen Variablen gehören zur Funktion. Benötigte Werte übergeben wir als Parameter, statt auf äußere Variablen zuzugreifen. Funktionen werden einmal definiert und können wiederholt aufgerufen werden. [6]

Strict Types wird jetzt sichtbar

```
quadrat(3);    // passt: int  
// quadrat("3"); // passt hier nicht: string statt int
```

Mit `declare(strict_types=1);` in unserer aufrufenden Datei löst der zweite Aufruf einen `TypeError` aus. Er bleibt deshalb auskommentiert. Ein deklarierter Rückgabetyt muss ebenfalls erfüllt sein. [3]

KEIN UNNÖTIGER UMWEG

Ein Vergleich liefert bereits `true` oder `false`. Eine Prüffunktion kann dieses Ergebnis direkt mit `return` zurückgeben. Ein zusätzliches `if`, das nur `true` oder `false` auswählt, ist dann nicht nötig. [4, 6]

BLOCK F / MINI-AUFGABEN 26-30

Eigene Funktionen

Kleine Berechnungen und Prüfungen mit return zurückgeben.

26 / Die erste eigene Funktion

1. Definiere die Funktion `begrueessung()` ohne Parameter mit Rückgabebetyp `string`.
2. Gib innerhalb der Funktion mit `return` den Text "Hallo PHP" zurück.
3. Rufe die Funktion außerhalb auf und gib ihr Ergebnis mit `echo` aus.

Erwartet: Hallo PHP

27 / Einen Parameter übergeben

1. Definiere `verdoppeln(int $zahl): int`.
2. Gib mit `return` das Doppelte von `$zahl` zurück.
3. Rufe die Funktion mit der Zahl 4 auf und gib das Ergebnis aus.

Erwartet: 8

28 / Zwei Parameter verwenden

1. Definiere `addiere(int $a, int $b): int`.
2. Gib die Summe der beiden Parameter zurück.
3. Rufe die Funktion mit 3 und 4 auf und gib das Ergebnis aus.

Erwartet: 7

29 / Eine String-Operation verpacken

1. Definiere `bereinigeName(string $name): string`.
2. Gib den mit `trim()` bereinigten Namen zurück.
3. Rufe die Funktion mit " Mia " auf und gib das Ergebnis aus.

Erwartet: Mia

30 / Eine Prüfung als bool zurückgeben

1. Definiere `istLeer(string $text): bool`.
2. Gib zurück, ob `trim($text)` exakt dem leeren String "" entspricht.
3. Teste mit `var_dump()` zuerst " " und danach "0".

Erwartet: `bool(true)`, danach `bool(false)`

Arbeite direkt im gleich nummerierten Abschnitt der Übungsdatei. Dort findest du auch einen kurzen Hinweis zu jeder Aufgabe.

MUSTERLÖSUNGEN / BLOCK A / AUFGABEN 01-05

Variablen und Kommentare

Die folgenden Ausschnitte stehen innerhalb einer PHP-Datei mit dem bereits gezeigten Grundgerüst. Die separat bereitgestellte Lösungsdatei enthält alle Startwerte und ist sofort ausführbar.

01 / Eine Variable anlegen

```
$kurs = "PHP";  
echo $kurs . PHP_EOL;
```

Ausgabe: PHP

02 / Bezeichner reparieren

```
$name1 = "Mia";  
$kursName = "PHP";  
echo $name1 . " lernt " . $kursName . PHP_EOL;
```

Ausgabe: Mia lernt PHP

03 / Kommentare schreiben

```
// Beispielort speichern  
$stadt = "Berlin"; /* Eine Textvariable */  
echo $stadt . PHP_EOL; # Ort ausgeben
```

Ausgabe: Berlin

04 / Einen Wert überschreiben

```
$punkte = 5;  
$punkte = 9;  
echo $punkte . PHP_EOL;
```

Ausgabe: 9

05 / Texte verbinden

```
$vorname = "Mia";  
$nachname = "Sommer";  
echo $vorname . " " . $nachname . PHP_EOL;
```

Ausgabe: Mia Sommer

MUSTERLÖSUNGEN / BLOCK B / AUFGABEN 06-10

Datentypen und Rechnen

06 / int und float ansehen

```
$anzahl = 3;  
$preis = 2.5;  
var_dump($anzahl, $preis);
```

Ausgabe: int(3) und float(2.5)

07 / string und bool unterscheiden

```
$zahlAlsText = "7";  
$aktiv = true;  
var_dump($zahlAlsText, $aktiv);
```

Ausgabe: string(1) "7" und bool(true)

08 / null und ein kleines Array

```
$hinweis = null;  
$farben = ["rot", "blau"];  
var_dump($hinweis, $farben[0]);
```

Ausgabe: NULL und string(3) "rot"

09 / Mit Variablen rechnen

```
$preis = 4;  
$anzahl = 3;  
$gesamt = $preis * $anzahl;  
echo $gesamt . PHP_EOL;
```

Ausgabe: 12

10 / Einen bekannten Zahlenstring umwandeln

```
$eingabe = "12";  
$zahl = (int) $eingabe;  
var_dump($zahl);
```

Ausgabe: int(12)

MUSTERLÖSUNGEN / BLOCK C / AUFGABEN 11-15

Entscheidungen

Die Klammern und Zweige bleiben in eigenen Zeilen lesbar.

11 / Eine negative Zahl korrigieren

```
$zahl = -4;
if ($zahl < 0) {
    $zahl = 0;
}
echo $zahl . PHP_EOL;
```

Ausgabe: 0

12 / Zwischen zwei Ausgaben wählen

```
$punkte = 7;
if ($punkte >= 5) {
    echo "Ziel erreicht" . PHP_EOL;
} else {
    echo "Weiter ueben" . PHP_EOL;
}
```

Ausgabe: Ziel erreicht

13 / Drei Fälle unterscheiden

```
$zahl = 0;
if ($zahl < 0) {
    echo "negativ" . PHP_EOL;
} elseif ($zahl === 0) {
    echo "null" . PHP_EOL;
} else {
    echo "positiv" . PHP_EOL;
}
```

Ausgabe: null

14 / Zwei Bedingungen verbinden

```
$punkte = 8;
$abgegeben = true;
if ($punkte >= 5 && $abgegeben === true) {
    echo "Bestanden" . PHP_EOL;
}
```

Ausgabe: Bestanden

15 / Eine Aktion mit switch auswerten

```
$aktion = "start";
switch ($aktion) {
    case "start":
        echo "Los" . PHP_EOL;
        break;
    default:
        echo "Unbekannt" . PHP_EOL;
}
```

Ausgabe: Los

MUSTERLÖSUNGEN / BLOCK D / AUFGABEN 16-20

Schleifen und kleine Arrays

16 / Mit for zählen

```
for ($i = 1; $i <= 3; $i++) {  
    echo $i . PHP_EOL;  
}
```

Ausgabe: 1, 2, 3 - jeweils in einer eigenen Zeile

17 / Mit while herunterzählen

```
$rest = 3;  
while ($rest > 0) {  
    echo $rest . PHP_EOL;  
    $rest--;  
}
```

Ausgabe: 3, 2, 1 - jeweils in einer eigenen Zeile

18 / do-while führt zuerst aus

```
$zahl = 5;  
do {  
    echo $zahl . PHP_EOL;  
    $zahl++;  
} while ($zahl < 3);
```

Ausgabe: 5 - genau einmal

19 / Arraywerte mit foreach lesen

```
$farben = ["rot", "gruen", "blau"];  
foreach ($farben as $farbe) {  
    echo $farbe . PHP_EOL;  
}
```

Ausgabe: rot, gruen, blau - jeweils in einer eigenen Zeile

20 / Mit Schlüssel und Wert verändern

```
$lager = ["Stift" => 2, "Heft" => 0];  
foreach ($lager as $artikel => $bestand) {  
    $lager[$artikel] = $bestand + 1;  
}  
echo $lager["Stift"] . " / " . $lager["Heft"] . PHP_EOL;
```

Ausgabe: 3 / 1

MUSTERLÖSUNGEN / BLOCK E / AUFGABEN 21-25

Strings verändern

21 / Äußere Leerzeichen entfernen

```
$name = "  Mia  ";  
$name = trim($name);  
echo $name . PHP_EOL;
```

Ausgabe: Mia

22 / In Kleinbuchstaben umwandeln

```
$text = "PHP MACHT SPASS";  
$text = strtolower($text);  
echo $text . PHP_EOL;
```

Ausgabe: php macht spass

23 / In Großbuchstaben umwandeln

```
$text = "hallo php";  
$text = strtoupper($text);  
echo $text . PHP_EOL;
```

Ausgabe: HALLO PHP

24 / Die Länge eines Textes bestimmen

```
$text = "Hallo";  
$laenge = strlen($text);  
echo $laenge . PHP_EOL;
```

Ausgabe: 5

25 / Einen Textteil ersetzen

```
$text = "Das Auto ist rot.";  
$text = str_replace("rot", "blau", $text);  
echo $text . PHP_EOL;
```

Ausgabe: Das Auto ist blau.

MUSTERLÖSUNGEN / BLOCK F / AUFGABEN 26-30

Eigene Funktionen

Jede Funktion wird einmal definiert. Die Zeilen darunter sind die zugehörigen Beispielaufrufe. Alle Aufrufe verwenden passende Datentypen.

26 / Die erste eigene Funktion

```
function begruessung(): string {  
    return "Hallo PHP";  
}  
echo begruessung() . PHP_EOL;
```

Ausgabe: Hallo PHP

27 / Einen Parameter übergeben

```
function verdoppeln(int $zahl): int {  
    return $zahl * 2;  
}  
echo verdoppeln(4) . PHP_EOL;
```

Ausgabe: 8

28 / Zwei Parameter verwenden

```
function addiere(int $a, int $b): int {  
    return $a + $b;  
}  
echo addiere(3, 4) . PHP_EOL;
```

Ausgabe: 7

29 / Eine String-Operation verpacken

```
function bereinigeName(string $name): string {  
    return trim($name);  
}  
echo bereinigeName(" Mia ") . PHP_EOL;
```

Ausgabe: Mia

30 / Eine Prüfung als bool zurückgeben

```
function istLeer(string $text): bool {  
    return trim($text) === "";  
}  
var_dump(istLeer("  "));  
var_dump(istLeer("0"));
```

Ausgabe: bool(true), danach bool(false)

NACHSCHLAGEN / QUELLEN

PHP-Handbuch

Die Erklärungen orientieren sich am offiziellen PHP-Handbuch. Die Beispiele und Mini-Aufgaben wurden für dieses PHP -Skript erstellt. Die Verweise [1] bis [6] führen zu den folgenden Themengruppen. Prüfung der Quellen: 29.09.2026.

[1] Syntax, Kommentare und Variablen

<https://www.php.net/manual/en/language.basic-syntax.php>

<https://www.php.net/manual/en/language.basic-syntax.comments.php>

<https://www.php.net/manual/en/language.variables.basics.php>

[2] Datentypen, Arrays und Ausgabe

<https://www.php.net/manual/en/language.types.intro.php>

<https://www.php.net/manual/en/language.types.array.php>

<https://www.php.net/manual/en/function.var-dump.php>

[3] declare und Strict Types

<https://www.php.net/manual/en/control-structures.declare.php>

<https://www.php.net/manual/en/language.types.declarations.php>

[4] Operatoren und Kontrollstrukturen

<https://www.php.net/manual/en/language.operators.arithmetic.php>

<https://www.php.net/manual/en/language.operators.comparison.php>

<https://www.php.net/manual/en/language.operators.string.php>

<https://www.php.net/manual/en/language.control-structures.php>

[5] Strings und Leerprüfungen

<https://www.php.net/manual/en/function.trim.php>

<https://www.php.net/manual/en/function.strtolower.php>

<https://www.php.net/manual/en/function.strtoupper.php>

<https://www.php.net/manual/en/function.strlen.php>

<https://www.php.net/manual/en/function.str-replace.php>

<https://www.php.net/manual/en/function.isset.php>

<https://www.php.net/manual/en/function.empty.php>

[6] Eigene Funktionen

<https://www.php.net/manual/en/functions.user-defined.php>

<https://www.php.net/manual/en/language.types.declarations.php>

VOR DEM NÄCHSTEN THEMENBLOCK

Du kannst nun Werte anlegen, Typen erkennen, einfache Entscheidungen und Schleifen schreiben, Texte verändern und kleine Funktionen aufrufen. Erst danach folgen Formulardaten und ausführlichere Prüfungen.